

1A) Refer to the X86-32 entry.S code in the lecture notes. We see two instructions:

```
cmpl $nr_syscalls,%eax // compare long => %eax - $nr_syscalls
```

```
jae syscall_badsys // jump if above or equal
```

These are essential for security. Imagine a world where these two instructions are missing. Explain how an attacker could exploit this as a security hole.

Ans: These instructions perform bounds checking to ensure the syscall number in %eax is valid before indexing the sys_call_table array. Without this check, an attacker could pass an out-of-bounds syscall number, causing the kernel to fetch and execute a function pointer from wrong memory locations in kernel mode, giving the attacker complete control of the system.

1B) For each of these scenarios (X86-32), explain if a new value for %esp is fetched from the TSS or not, or if the scenario is impossible. Justify your answer!

1B1) We are running a user-mode program when the PIT interrupt arrives.

The CPU needs to switch from a user mode to kernel mode since the PIT interrupt happened. Therefore, the cpu fetches the kernel stack pointer from the TSS.

1B2) During handling of a system call, a disk I/O completion interrupt arrives.

This is a transition from kernel mode to kernel mode. Thus, the %esp is already pointing to the kernel stack, which means the CPU doesn't need to fetch new one from the TSS.

1B3) While handling an interprocessor interrupt (IPI), a page fault occurs.

Impossible.

The Linux kernel is coded to never produce page faults during interrupt handling. IPI is an interrupt handler, and kernel code executing in interrupt will never cause a page fault. The only exception where a page fault can occur in kernel mode is during system call handling. Thus, this case is impossible.

1C) The root filesystem (volume) must be mounted before the kernel begins user-mode process #1 (init or systemd) since the executable for that process is in the filesystem. What would happen if the root filesystem can't be mounted by the kernel, e.g. because of a disk I/O error, or misconfiguration? This question requires some research. The answer does not appear directly in the lecture notes.

The boot process halts with a kernel panic because the kernel cannot execute `/sbin/init` (or `systemd`) to create Process #1, which is required to start all user-mode processes. Without the root filesystem being mounted at the beginning, the kernel works fine but cannot interact with user programs. The system cannot proceed, and manual intervention is required to fix the problem and reboot.

1D) Why does the `TIF_NEED_RESCHED` flag exist?

The `TIF_NEED_RESCHED` flag exists to enable deferred preemption at safe points. When the scheduler determines another task should run, it cannot immediately switch while in critical kernel code, so it sets this flag instead. The flag is checked when returning to user mode, where it triggers `schedule()` to perform the context switch. This prevents race conditions and data corruption.